1 Summary

In this homework, you will learn how to solve RL problems using well-known algorithms. You will write and submit a report containing your answers and figures. For each part, please provide a description of your solution and any corresponding plots. **In addition, for each part, please address the following two tasks**:

- **Edge cases.** What edge cases may arise from the problem and how did you handle them? Please document edge cases of the problem that you considered and explain how you ensured that your solution has handled them correctly.
- **Tests for validating the correctness of your algorithm.** Beyond asking AI, how do you know that your algorithm is working correctly? Test whether your implementation is behaving as predicted. For example, you may change certain parameters or the structure of the problem and see if the output matches your expectation. You may also reduce the stochasticity of the environment or policy so that the process is close to deterministic. You may also find it useful to isolate the first 10 steps and observe how the states, values, rewards, and policy interact at each step.

Please provide directions and documentation for how to execute validation and test experiments (what does each test do, how does it validate, etc). Provide quantitative and convincing evidence that you have sufficiently addressed the above questions. Do not confuse these requirements with unit tests or basic bug fixes, for example checking if the data is in the right format. Instead, please use creativity, critical thinking, and your understanding of RL/ML/AI to determine if the algorithm is appropriately tuned and correctly handling the specific underlying task. Finally, answer the following question: **Given an additional 3 months (extended period), what other validation tests and experiments would you perform?**

Submission Guidelines. Please submit on Canvas a pdf report with the following:

- Your GitHub username, repository name and **link to the GitHub repository**;
- Instructions of how to run your code;
- Labeled and formatted plots required for each part;
- Responses to all discussion questions.

Please provide in your GitHub repository all code used for this assignment. **If your submission does not follow the provided guidelines, a significant number of points will be deducted from your score.** In particular, each submission should clearly provide the GitHub link to the source code for the assignment.

2 Value and Policy Iteration (30 points)

Consider the shuttle dispatching problem, where shuttles are sent to move passengers waiting at a station. At each time step/decision epoch, an agent decides whether or not to dispatch a shuttle depending on the number of customers waiting for the service. A standard shuttle dispatch problem has the following parameters:

- K = The capacity of a shuttle if it is dispatched,
- A_t = A random variable giving the number of customers arriving during time interval t.
- c_f = The cost of dispatching a shuttle.
- c_h = The cost per customer left waiting per time period (incurred after dispatch).

Given $K = 15$, $c_f = 100$, $c_h = 2$ and assuming that A_t follows $Unif\{1, 2, 3, 4, 5\}$, model the problem as a Markov Decision Process and solve it using:

a) Enumeration (with time period $T = 500$). Plot the optimal value function at time 0 versus number of customers waiting.

b) Value iteration ($T = \infty$). Plot optimal value function versus number of customers waiting.

c) Policy iteration. Plot optimal policy versus number of customers waiting.

You can assume that number of people in station cannot exceed 200. Use discount rate $\gamma = 0.95$. If you make any other assumption, please state it in your report.

3 Policy Gradient: Part 1 (40 points)

Using PyTorch, implement the basic policy gradient algorithm for the following environments from OpenAI Gym. You need to train a simple neural network that models the policy and takes actions based on the provided states. In each of the games mentioned below, the objective for the agent is to win the game or maximize the total score achieved during the game. To optimize the policy parameters, set the the discounted reward over an episode as your objective function, and use Adam as your gradient optimizer with learning rate 0.01. Feel free to use a different learning rate if you find that training is more stable and mention it in your report.

For both parts below, plot the episode reward against the number of training episodes. Overlay this plot with the moving average of the rewards over 100 episodes. When you finish training, roll out 500 episodes using your trained model. Plot the histogram of the 500 episode rewards and report the mean and the standard deviation.

a) **CartPole-v1.** Take a look here to learn how the game is played. Use discount factor = 0.95.

b) **Pong-v5.** Take a look here to learn how the game is played. Use discount factor of 0.99. Use frames of the game as input to a neural net which models the policy. The game has six actions [NOOP, FIRE, RIGHT, LEFT, RIGHTFIRE, LEFTFIRE], but you should train a network that chooses the best action only between [RIGHT, LEFT] (actions 2 and 3).

You may pre-process all the images from the game with following function before feeding them into the neural network.

```
def preprocess(image):
  """ prepro 210x160x3 uint8 frame into 6400 (80x80) 2D float array """
  image = image[35:195] # crop
  image = image[::2,::2,0] # downsample by factor of 2
  image[image == 144] = 0 # erase background (background type 1)
  image[image == 109] = 0 # erase background (background type 2)
  image[image != 0] = 1 # everything else (paddles, ball) just set to 1
  return np.reshape(image.astype(np.float32).ravel(), [80,80])
```

Notes:

- You are required to use the specified version of the game environments, i.e. CartPole-v1 and Pong-v5, for both questions.
- If you are using deepdish, you must use a single GPU card.

Policy Gradient: Part 2 (20 points)

You have implemented the policy gradient algorithm to learn how to play the two games. However, the training suffers from high variance of the rewards in the sampled episodes. Using a baseline helps to reduce the variance. In Part 2, repeat the experiments of Part 1, now using a constant or non constant baseline of your choice. Explain which baseline you have chosen and how you implemented it.

4 Grading

The grading breakdown will be as follows. In addition, solutions will be generally graded based on correctness, as demonstrated by various components.

- Value and Policy Iteration: 30 points
  - Edge cases and tests: 15 points

  - Plots: 15 points

- Policy Gradient Part 1: 40 points
  - Edge cases and tests: 20 points
  - Plots: 20 points
- Policy Gradient Part 2: 20 points
  - Edge cases and tests: 10 points
  - Plots: 10 points
- Report formatting and readability: 10 points